

RO 2-056 6.11.2 [basic.contract.eval] Make Contracts Reliably Non-Ignorable

Document number: P3911R2

Date: 2026-01-14

Authors: Darius Neațu <dariusn@adobe.com>, Andrei Alexandrescu <andrei@nvidia.com>, Lucian Radu Teodorescu <lucteo@lucteo.ro>, Radu Nichita <radunichita99@gmail.com>, Herb Sutter <herb.sutter@gmail.com>

Audience: EWG

Contents

Abstract

Revision History

Acknowledgments

TL;DR

§1. Introduction

- §1.1 Motivation
- §1.2 Overview of R1 revision: Following the EWG Kona 2025 direction

§2. Scope

- §2.1 Design constraints per EWG mandate
- §2.2 Key open questions addressed by this paper

§3. Proposal: Add source syntax for minimal always-enforced contract assertions

- §3.1 Existing practice
- §3.2 Prior work
- §3.3 ABI implications

§4. Design choice 1: Should "always-enforced" contract assertions **support enforce and/or quick-enforce semantics**?

- §4.1 Enforcement option 1: Support "always-contract-terminate," and leave calling a violation handler implementation-defined
- §4.2 Enforcement option 2: Support both `enforce` and `quick-enforce` contract semantics

§5. Design choice 2: What **syntax** should be used?

- §5.1 Syntax option 1: Use `<label>`
- §5.2 Syntax option 2: Use `!` (bang)
 - §5.2.1 `!` (bang) existing practice

§6. Conclusion

§7. Wording

§8. Frequently Asked Questions

- §8.1 Why not a syntax such as `enforce_pre` or `pre_always` ?
- §8.2 Why not use a terse syntax with a symbol other than `!` , such as `pre#` or `pre@` ?
- §8.3 Why not do this only for `pre` , leaving `post` and `contract_assert` unchanged?
- §8.4 What about support for indirect calls (pointers to functions and virtual functions)?
- §8.5 How will this change evaluation semantics?
- §8.6 How does this proposal impact the Standard Library?
- §8.7 Does this paper address all the questions raised in P3912R0 [5], P3919R0 [6], and P3946R0 [7]?
- §8.8 What about requiring header dependency?
- §8.9 Will "always-enforced" contract assertions behave differently from the "normal" ones?
- §8.10 What does it mean that "execution will not continue past the contract assertion" when an "always-enforced" contract assertion is violated?
- §8.11 How does tooling benefit from this proposal?
- §8.12 Does this proposal change the semantics of the C++26 Contracts facility?
- §8.13 Is the predicate of an "always-enforced" contract assertion evaluated exactly once?
- §8.14 Is there any previous implementation experience?

Abstract

Building on motivation from EWG Kona 2025, this paper proposes a minimal pure extension to the C++26 Contracts facility. This extension addresses the Romanian National Body's (NB) concern by enabling the writing of an individual contract assertion that guarantees the program execution will not continue past the contract assertion if it is violated, regardless of the semantics of other contract assertions. This addition to the C++26 Contracts facility allows reliable use of enforced contract assertions in large codebases with mixed contract semantics and also in hardened libraries.

This paper's narrow extension is both necessary and sufficient to address the Romanian NB's concern. It does not preclude, nor co-require, proposals that add properties to "normal" or "always-enforced" contract assertions (such as no-constification, single evaluation, or similar properties) that may achieve consensus in EWG.

Revision History

- P3911R2 (<https://wg21.link/P3911R2>) (January 2026, after EWG telecon January 8th)
 - Section §1 (introduction): extend the motivation (see full section §1.1).
 - Section §3 (proposal): add sections §3.2 (prior work) and §3.3 (ABI implications).
 - FAQs updates:
 - Add explicit statement about *not* requiring header dependency (see full section §8.8).
 - Add new FAQs: §8.11 (tooling), §8.12 (unchanged semantics), §8.13 (predicate evaluation count), §8.14 (previous implementation experience).
 - Various editorial fixes.
- P3911R1 (<https://wg21.link/P3911R1>) (December 2025, after Kona Meeting)
 - Focus on implementing the EWG mandate: see sections §1.2 and §2-§5.
 - Add Acknowledgments section.
 - Add TL;DR section.

- Temporarily remove content in section 7. Wording until EWG agrees on design directions in telecons.
- Add §8 Frequently Asked Questions section.
- P3911R0 (<https://wg21.link/P3911R0>) [1] (November 2025, Kona Meeting)
 - Surface the issue from Romanian NB comment RO 2-056 6.11.2 [basic.contract.eval] Make contract assertions reliably non-ignorable (<https://github.com/cplusplus/nballot/issues/631>).
 - Iterate through all four possible solutions (labeled as v1 (<https://wg21.link/P3911R0#v1-Add-pre-minimal-always-enabled-contracts>), v2 (<https://wg21.link/P3911R0#v2-Remove-ignore-Semantics>), v3 (<https://wg21.link/P3911R0#v3-Add-full-Contracts-Properties-Framework>), v4 (<https://wg21.link/P3911R0#v4-Move-C-Contracts-to-Other-Shipping-Vehicle>)), which should be considered in EWG.

Acknowledgments

We thank everyone who provided feedback on drafts of this paper, in particular: Gašper Ažman, Joshua Berne, Peter Bindels, Timur Doumler, Andrei Durlea, Pablo Halpern, Dan Katz, Thomas Köppe, Andrzej Krzemieński, Brad Larson, Lisa Lippincott, Jens Maurer, Jonas Persson, Nina Dinka Ranns, Oliver Rosten, Iain Sandoe, David Sankel, Adrian Stanciu, Ville Voutilainen.

TL;DR

Under the current C++26 Contracts design, if a contract assertion condition is false, the resulting behaviour depends on the evaluation semantics as follows:

- `ignore` — continues execution;
- `observe` — calls the handler and continues execution;
- `quick-enforce` — terminates execution immediately;
- `enforce` calls — the handler and terminates execution, unless an exception is thrown in the handler.

By design, `pre`, `post`, and `contract_assert` can use any of these semantics. The implementation determines which evaluation semantics apply to each evaluation of the contract assertion.

This paper presents two alternative extensions to the C++26 Contracts facility, either of which is sufficient to address the Romanian NB Comment:

1. Introduce "hardened" variations of `pre`, `post`, and `contract_assert` that may only use `quick-enforce` or `enforce` semantics. It is implementation-defined which of these two semantics is used for each contract assertion evaluation. Syntax: `pre<terminating>` or `pre!`.
2. Introduce "always-enforce" and "always-quick-enforce" variations of `pre`, `post`, and `contract_assert` that may only use `enforce` or `quick-enforce` semantics, respectively, for each contract assertion evaluation. Syntax: `pre<enforce>` or `pre!` for "always-enforce"; and `pre<quick_enforce>` or `pre!!` for "always-quick-enforce".

1. Introduction

1.1 Motivation

```
// Syntax example for always enforced preconditions – pre! (bang)
int critical_access(int* p) pre!(p) { return *p; }
```

This paper is motivated by concerns raised in RO 2-056

(<https://github.com/cplusplus/nbballot/issues/631>), which highlight the risks of relying on enforcement semantics selected post-definition. Although users should not generally assume that contract assertions are always enforced (depending on project configuration and developer awareness), the core issue is more fundamental: in large-scale, real-world codebases, some contract assertions represent invariants that are sufficiently critical that execution must not continue if they are violated. Under the current C++26 model, such invariants cannot be expressed solely as contract assertions, because enforcement may be disabled or weakened by configuration.

This proposal strengthens the C++26 Contracts facility (introduced by P2900R14) by virtue of three key properties that address related but distinct concerns: monotonicity, composability, and local reasoning.

The proposed rule is *monotonic* because it only strengthens enforcement guarantees; it never weakens existing guarantees and does not introduce additional permitted executions. Programs that do not opt in are unaffected.

Additionally, the proposal is *composable* because a module can state that a given assumption must never be ignored and have that guarantee preserved across translation units, build modes, link boundaries, and runtime configurations.

By *local reasoning* we mean semantic guarantees expressed locally in source code. A contract assertion that may or may not be enforced is distinguishable from a comment only after auditing environmental components such as build configuration and other toolchain settings. For some invariants (e.g., security- and safety-critical invariants), non-enforcement constitutes a defect rather than a configuration choice. This proposal allows authors to state which assumptions are semantic requirements rather than configurable hints.

Absent such a facility, users must resort to duplicating logic:

```
int transmogrify(int x, int y, double z)
{
    pre(x > 0)
    pre(y != 0)
    pre(!std::isnan(z))
    {
        if (x <= 0 || y != 0 || std::isnan(z)) {
            cerr << "Fatal contract violation.\n";
            abort();
        }
        ...
    }
}
```

Such code repeats vital checks once as contract assertions for specification and tooling purposes, and again as ordinary control flow to guarantee termination when said requirements are not met. This duplication is error-prone, inefficient, bulky at source and binary level, prone to divergence (as illustrated by the inconsistency in the example above — the `if` statement should test `y == 0`), and corrosive to *the use of contracts to express program correctness*.

Because the contract assertions's evaluation semantics are intentionally not observable from source code, performance-conscious developers cannot elide the `if` in `transmogrify` even if the runtime environment is configured to always enforce the contract assertion. As one workaround begets another, a project-specific trick could be to define and maintain a flag that informs source code whether contracts are always enforced (subject to well-known liabilities, including non-portability, configuration drift, and source-level coupling to build policy):

```
int transmogrify(int x, int y, double z)
{
    pre(x > 0)
    pre(y != 0)
    pre(!std::isnan(z))
    {
        // Project-specific flag approximating contract enforcement state
        #if !CONTRACTS_ALWAYS_ENFORCED
            if (x <= 0 || y != 0 || std::isnan(z)) {
                cerr << "Fatal contract violation.\n";
                abort();
            }
        #endif
        ...
    }
}
```

All of the issues discussed above are eliminated by a monotonic, opt-in enforcement guarantee with local reasoning.

The initial (P3911R0 (<https://wg21.link/P3911R0>) [1]) version of this paper discussed four potential solution approaches:

1. Add a way to write a contract assertion that will always be enforced in all cases (regardless of the currently active contract semantics).
2. Remove "ignore" (and, as was intended, "observe") contract semantics from the C++ Working Draft.
3. Adopt all of P3400R2 Controlling Contract-Assertion Properties (<https://wg21.link/P3400R2>) [3] — the full C++ Contracts properties framework.
4. Remove the C++26 Contracts facility from the C++ Working Draft introduced by P2900R14 (<https://wg21.link/P2900R14>) [2].

The R0 revision of this paper recommended pursuing option 1. It is purely additive to C++26 Contracts, pulls forward into C++26 one specific capability from the post-C++26 direction described in P3400R2 that addresses the current concerns, and does so in a forward-compatible way with P3400R2.

1.2 Overview of R1 revision: Following the EWG Kona 2025 direction

In Kona 2025, EWG encouraged work (<https://github.com/cplusplus/nbballot/issues/631#issuecomment-3488277609>) in the direction proposed in D3911R0 (now P3911R0 (<https://wg21.link/P3911R0>) [1]):

Poll: RO 2-056 6.11.2 [basic.contract.eval] Make contracts reliably non-ignorable — we want to pursue a change for C++26 as proposed in one of the four options listed in D3911R0.

SF	F	N	A	SA
18	25	22	7	0

Result: consensus for change

Option 1, which proposes making contracts reliably non-ignorable by adding a syntax that ensures contract assertions have guaranteed effects, was the main focus of the EWG discussion.

EWG raised three important additional observations regarding Option 1:

- It is a **pure extension** that does not affect existing C++26 Contracts semantics and pulls forward a **specific narrow capability** already considered and encouraged in P3400R2 for post-C++26 Contracts evolution.
- It may also **address other NB concerns**, notably enabling the implementation of the C++26 hardened standard library using C++26 Contracts. (For example, via `#ifdef` guards, such as `#ifdef __hardening / pre<terminating>(idx < size()) / #endif ;` today, libraries that do this put nonstandard code in the guarded section.)
- It should apply uniformly to **all three contract assertions** — `pre` , `post` , and `contract_assert` — as requested by several EWG participants. This ensures consistent capability across all contract assertion types.

2. Scope

2.1 Design constraints per EWG mandate

To follow EWG direction, increase consensus, and avoid scope creep, this revision adopts the following design constraints:

1. **Scope: Option 1 only.** We have removed options 2, 3, and 4, which EWG deemed as unviable.
2. **Pure extension.** This paper proposes a pure extension to the C++26 Contracts facility from the post-Kona C++ Working Draft. It does not change the existing semantics of `pre(condition)`, `post(ret: condition)`, or `contract_assert(condition)`.
3. **Minimal semantics delta.** This paper does not propose additional changes beyond providing a way to spell "always-enforced" contract assertions in source code, which is enough to satisfy the Romanian NB's concern and which EWG encouraged. To avoid scope creep, it does not propose additional properties of "always-enforced" contract assertions (e.g., no-constification, single evaluation, or similar properties) that differ from existing C++26 Contracts and may achieve EWG consensus independently. The Romanian NB expresses no position on the inclusion of such additional properties.

See §8 Frequently Asked Questions for further discussion of additional design questions.

2.2 Key open questions addressed by this paper

We identify the following open questions:

Q1. Should "always-enforced" contract assertions **support enforce and/or quick-enforce semantics**? See section §4.

Q2. What **syntax** should be used? See section §5.

3. Proposal: Add source syntax for minimal always-enforced contract assertions

This paper proposes introducing a **minimal syntax to mark certain contract assertions as "always-enforced" in source code (see §4)**. This additional syntax is placed between the contract keyword and the condition (see §4 and §5 for the proposal):

Case	Syntax
Unchanged syntax	<code>pre(x != 0)</code>
Added "always-enforced" syntax	<code>pre <i>placeholder-tokens</i> (y != 0)</code>

A " `pre placeholder-tokens (condition)` " assertion is always enforced, independent of any other contract semantics selected for `pre (condition)` .

The same syntax choice applies to `post` and `contract_assert` .

This design represents a balanced compromise: it preserves configurability while ensuring selected contract assertions are reliably checked. No future extensions are precluded.

3.1 Existing practice

There is a substantial body of existing practice that motivates "always-enforced" contract assertions, demonstrates their utility, and makes the model familiar to users. Many production codebases ship an always-on assertion (separate from `assert` , which is sometimes-on by means of the `NDEBUG` macro).

Such mechanisms, commonly named `CHECK` , `VERIFY` , or `ALWAYS_ASSERT` in existing codebases, provide run-time validation of key conditions even in production: they verify the condition and do not continue execution if it does not hold. They are widely used as safety checks for preconditions, i.e., as intentionally redundant code that has no observable effect in correct program executions, but deterministically prevents execution from continuing in incorrect executions. Such mechanisms may also be used outside an assertion context, where they are not redundant.

Below, we present a selection of representative, widely used examples that leverage non-continuing (terminating) enforcement semantics in production:

- **Google Abseil's CHECK** (<https://abseil.io/docs/cpp/guides/logging#CHECK>) ("terminates the process" in all builds; paired with debug-only `DCHECK`) is used in a plethora of Google and third-party projects, notably including **Protocol Buffers**, **glog**, **gRPC**, **Chromium**, and **TensorFlow**.
- **Facebook Folly's XCHECK** (<https://github.com/facebook/folly/blob/main/folly/logging/xlog.h>) (always-on — "this crashes the program").
- **WebKit's RELEASE_ASSERT** (<https://github.com/WebKit/WebKit/blob/main/Source/WTF/wtf/Assertions.h>) ("crashes with info"; paired with `ASSERT`).
- **LLVM's cantFail** (https://llvm.org/doxygen/llvm_2Support_2Error_8h.html) (several overloads — "report a fatal error").
- **Mozilla Gecko's MOZ_RELEASE_ASSERT** (<https://github.com/mozilla/gecko-dev/blob/5836a062726f715fda621338a17b51aff30d0a8c/mfbt/Assertions.h#L566>).
- **Apache Arrow's ARROW_CHECK**
(<https://github.com/apache/arrow/blob/3dc982183db45ced57ef1565e022dcb647a00745/cpp/src/arrow/util/logging.h#L67>).

Note: The quoted phrases above are taken verbatim from the respective project documentation.

Experience shows that defining a project-specific assertion facility (or using an established one such as Abseil or Folly) is a mark of project growth and maturity. Such facilities almost always include an API that deterministically terminates execution when a boolean condition evaluates to false.

Externally-controlled assertions and always-active checks rely on the same underlying mechanics, which is why mature codebases commonly pair them: a debug-only assertion alongside an always-fatal check for essential invariants. Absent a reliable facility that terminates execution when a condition is false, users must duplicate logic by first writing an assertion that may or may not be active, then adding manual code to test the condition again and terminate on failure. A single, always-available check facility avoids that duplication, keeps intent obvious, and reduces divergence across configurations.

3.2 Prior work

The following papers provide relevant prior work and historical context for "always-enforced" contract assertions:

- P3640R0: Make contracts reliable by default (<https://wg21.link/P3640R0>) [8].
- P1429R3: Contracts That Work (<https://wg21.link/P1429R3>) [9].
- P1332R0: Contract Checking in C++: A (long-term) Road Map (<https://wg21.link/P1332R0>) [10].

Existing implementation experience for these proposals is discussed in section §8.14.

3.3 ABI implications

The C++26 Contracts facility is designed to preserve ABI backward-compatibility when adding function contract specifiers. P2900R14 (<https://wg21.link/P2900R14>)'s Principle 16 ("No ABI Break") states: *"A conforming implementation should be able to guarantee that adding function contract specifiers to an existing function preserves ABI backward-compatibility."*

This ensures that code can add function contract specifiers incrementally without requiring recompilation of client code. P2900R14 does not explicitly discuss the effects of changing or removing existing function contract specifiers.

It is this particular scenario, namely removing an "always-enforced" contract assertion, which deserves special attention. Consistent with P2900R14's design intent, this proposal takes a conservative and future-proof approach and stipulates that changing or removing a function's contract specifiers is likewise intended not to require recompilation of already-compiled callers.

To ensure no ABI break, callers are not permitted to assume control flow does not continue if a `pre!` or `post!` function contract specifier is violated. Consider:

```
// Using one of the proposed syntaxes as example.
int fun(int x) pre!(x != 0);

// gun() - perhaps located in a different library.
int gun(int y) {
    if (y == 1) {
        fun(y - 1);
        std::printf("Hello, world!\n");
    }
    return 0;
}
```

If `pre!(x != 0)` were a semantic guarantee that execution cannot continue past the call, then an optimizing compiler could assume that execution does not continue execution past the call `fun(y - 1)` and eliminate the call to `printf` entirely because it's on an unreachable code path. If later on the owner of `fun` changes its declaration to use `pre` instead of `pre!` (or remove the function contract specifier entirely), `gun` must be recompiled; otherwise, the program may exhibit undefined behavior due to assumptions made during caller's compilation that are no longer valid.

In order to stay true to the design goals of the C++26 Contracts facility, this proposal stipulates that `pre!` is **an enforcement, not an assumption**. An implementation is not permitted, for the purposes of caller-side compilation, to treat violation of an "always-enforced" contract assertion as implying that execution cannot continue past a function call. For all purposes relevant to caller-side compilation, `pre!` shall be treated equivalently to `pre`, and `post!` equivalently to `post`.

This choice is intentionally conservative with respect to ABI stability and implementation freedom. It benefits from reusing the extensive analysis of P2900R14 and leaves room for future refinements. Also, this proposal does not mandate where enforcement is performed (caller or callee), preserving the implementation latitude envisioned by P2900R14.

4. Design choice 1: Should "always-enforced" contract assertions support enforce and/or quick-enforce semantics?

For this design choice, the following major options were considered:

- Support only one of the "enforce" or "quick-enforce" contract semantics?
- Support both explicitly selectable in the source code?
- Do what a hardened standard library does (i.e., leave it implementation-defined)?

During the EWG discussion and on-time papers in Kona, such as P3835R0

(<https://wg21.link/P3835R0>) [4], it appears there are three large, partially overlapping groups:

- those who want to specify "always enforce" semantics in source code;
- those who want to specify "always quick-enforce" semantics in source code; and
- those who want to specify "never ignore or observe, without a preference for enforce or quick-enforce" contract semantics in source code.

Either option presented in sections §4.1 or §4.2 (see below) would satisfy the Romanian NB's concern. More generally, if EWG identifies another option with consensus support that guarantees program execution does not continue (see §8.10) when that contract assertion is violated (including, but not limited to, never being subject to ignore or observe semantics), the Romanian NB would accept that as satisfactory.

4.1 Enforcement option 1: Support "always-contract-terminate", and leave calling a violation handler implementation-defined

This option makes a single **"always contract-terminate" syntax** selectable in source code, leaving it implementation-defined whether a contract-violation handler is invoked.

See §5 for *placeholder* syntax options.

Case	Syntax
Unchanged	<code>pre (x != 0)</code>
Added "always-contract-terminated" syntax	<code>pre <i>terminate-tokens</i> (y != 0)</code>

A " `pre terminate-tokens (condition)` " shall always be enforced regardless of other semantics that may be used for `pre (condition)` , and the implementation shall define whether or not a contract-violation handler is called (a terminating semantics is guaranteed to be used).

Note that the same design choice applies to `post` and `contract_assert` .

4.2 Enforcement option 2: Support both `enforce` and `quick-enforce` contract semantics

This option provides **two distinct placeholders**, allowing the programmer to express **either "always enforce" or "always quick-enforce"** in source code for a given contract assertion.

Case	Syntax
Unchanged	<code>pre (x != 0)</code>
Added "always-enforced" syntax	<code>pre <i>enforce-tokens</i> (y != 0)</code>
Added "always-quick-enforced" syntax	<code>pre <i>quick-enforce-tokens</i> (y != 0)</code>

A " *pre enforce-tokens* (condition) " is always enforced, and a " *pre quick-enforce-tokens* (condition) " is always quick-enforced, regardless of any other contract semantics selected for *pre* (condition) .

Note that the same design choice applies to *post* and *contract_assert* .

5. Design choice 2: What syntax should be used?

Several alternatives were considered. This paper proposes two concise syntax choices (§5.1 and §5.2 — see below) that are forward-compatible with future directions, such as P3400R2 (<https://wg21.link/P3400R2>), without precluding future library-based assertion-control mechanisms (see §8.8).

Either option presented in sections §5.1 or §5.2 (see below) would satisfy the Romanian NB’s concern. More generally, if EWG identifies another option that has consensus support, the Romanian NB would accept that, provided that the contract semantics guarantee the program code does not continue execution after a violation (see sections §4 and §8.10).

The Romanian NB requests only a narrow extension that allows a contract assertion to be marked as enforced in source code. Instead of inventing a new syntax, this paper follows the design direction previously discussed and encouraged in EWG.

5.1 Syntax option 1: Use `<label>`

This option follows the current syntax of proposed P3400R2 (<https://wg21.link/P3400R2>).

Under **Enforcement option 1** (see §4.1), use `<terminating>` (or other similar label - e.g., `<always_terminating>` , `<hardened>` etc) to express a contract assertion that is **always** evaluated using terminating semantics (per §4.1, with contract-violation handler invocation left implementation-defined):

Contract assertion	"always contract-terminate" syntax
Precondition	<code>pre <terminating> (condition)</code>
Postcondition	<code>post <terminating> (ret: condition)</code>
Assertion statement	<code>contract_assert <terminating> (condition)</code>

Under **Enforcement option 2** (see §4.2), which provides two distinct syntactic forms, use the labels `<enforce>` and `<quick_enforce>` (or other similar labels - e.g., `<always_enforce>` and `<always_quick_enforce>` , etc) to express a contract assertion that is **always** evaluated using the "enforce" and "quick-enforce" semantics, respectively:

Contract assertion	"always-enforce" syntax	"always-quick-enforce" syntax
Precondition	pre <i><enforce></i> (condition)	pre <i><quick_enforce></i> (condition)
Postcondition	post <i><enforce></i> (ret: condition)	post <i><quick_enforce></i> (ret: condition)
Assertion statement	contract_assert <i><enforce></i> (condition)	contract_assert <i><quick_enforce></i> (condition)

5.2 Syntax option 2: Use **!** (bang)

This option follows the original strawman syntax in R0 of this paper (see P3911R0 (<https://wg21.link/P3911R0>)).

Under **Enforcement option 1** (see §4.1), use **!** to express "terminating" semantics (with contract-violation handler invocation left implementation-defined):

Contract assertion	Syntax
Precondition	pre ! (condition)
Postcondition	post ! (ret: condition)
Assertion statement	contract_assert ! (condition)

Under **Enforcement option 2** (see §4.2), which has two syntaxes, use **!** and **!!** to express a contract assertion that is evaluated using the "enforce" and "quick-enforce" semantics, respectively:

Contract assertion	Always enforce syntax	Always quick-enforce syntax
Precondition	<code>pre ! (condition)</code>	<code>pre !! (condition)</code>
Postcondition	<code>post ! (ret: condition)</code>	<code>post !! (ret: condition)</code>
Assertion statement	<code>contract_assert ! (condition)</code>	<code>contract_assert !! (condition)</code>

We believe this style is forward-compatible with P3400R2 because "terminating" semantics is likely a common use case that warrants a convenient shorthand; if we had P3400R2-style contract assertion labels today, "always enforce/quick-enforce this contract assertion" would still benefit from a dedicated terse shorthand to keep common usage simple and ergonomic.

5.2.1 ! (bang) existing practice

This follows existing use of the `!` suffix to connote "do not continue execution" (with halt or throw semantics) in other popular languages that also use `!` as a prefix logical-not operator, and this notation is widely understood and does not confuse programmers. For example:

- TypeScript: `value!.prop` is a non-null assertion.
- Swift: `let x = optional!` is a forced unwrap with assert semantics, where if the optional is `nil` the program terminates.
- Kotlin: `val x = nullable!!` is a non-null assertion.
- Elixir: `File.read!/1` is a "bang function" that raises an exception on error.
- Ruby: `update!` and `save!` are idiomatic names for methods that mutate state or raise on failure.

The following languages use `!` as a prefix logical-not operator and also use `!` as suffix with a distinct, unrelated meaning, again without causing confusion:

- Julia: `sort!(v)` is a mutating-function convention.
- Rust: `println!("hi")` is a macro invocation, and `!` is a "never" type (when written in a grammar position where a type would appear).
- D: `Tuple!(int, string)` is a template instantiation.

A concern has been raised that `!` might be interpreted as "logical NOT this precondition."

This concern has two aspects:

- That `!` as a prefix and as a suffix have different meanings. Evidence from experience with the languages listed above suggests this is not a significant problem.
- That this specific option permits writing `pre!(condition)` and `pre(!condition)` with distinct meanings. It is noted that C++ permits syntax such as `if (!x > 1)` and `if (!(x > 1))` without this being a known source of bugs as reflected in books, articles, "gotcha" lists, and coding standards documents.

One drawback is that this allows writing `pre not` using an alternative token. The C++ language already has several precedents: the alternative token `compl` for `~` can be used in the destructor declaration syntax; the alternative token `bitand` for `&` can be used in reference declarators and as an address-of operator; and the alternative token `and` for `&&` can be used to construct rvalue reference types. Spelling a destructor declaration as `compl Type()`, a function declaration as `void f(int and a, int bitand b)`, or the address of an object as `bitand a` (see <https://godbolt.org/z/x7EY1Eoh3> (<https://godbolt.org/z/x7EY1Eoh3>)) may be an amusing curiosity but not a source of practical problems.

6. Conclusion

This paper proposes a small, purely additive extension to the C++26 Contracts facility that allows a contract assertion to be marked as "always-enforced". By enabling a simple source-level choice among `enforce`, `quick-enforce`, or `terminating` semantics, the proposal improves the practical reliability of contract assertions in large codebases and hardened libraries, while preserving the existing design and helping build consensus around the C++26 Contracts facility and the C++ Working Draft.

7. Wording

TBD — R0 had initial wording, which is now removed; we'll add an updated one when EWG decides about the final design (multiple design options are presented in this paper).

8. Frequently Asked Questions

8.1 Why not a syntax such as `enforce_pre` or `pre_always` ?

Several syntax alternatives were considered.

This paper proposes `pre<label>` style and `pre!` style because these two syntaxes are **long-term forward-compatible with P3400R2**, which has already been encouraged in EWG, and therefore aligns with the EWG Kona 2025 direction to bring forward to C++26 this narrow previously proposed capability.

Name-based syntaxes such as `enforce_pre` or `pre_always` would not naturally compose with future generalized contract assertion annotations envisioned by P3400R2.

- The `pre<label>` style is directly forward-compatible with P3400R2 because it follows the same syntax direction. If P3400R2 were available today, this is the syntax that would be used.
- The `pre!` style is also forward-compatible with P3400R2 because always enforcing is expected to be a common use case that warrants a concise shorthand; even if P3400R2 were available today, "always enforce this contract assertion" would still benefit from a dedicated, terse shorthand.

This paper does not propose alternative syntaxes because they would not be forward-compatible with P3400R2, which aligns with EWG guidance for this proposal. If P3400R2 were available, EWG would be unlikely to adopt an additional spelling that would be redundant and as verbose as `pre<label>`.

However, this proposal is open to adopting any syntax choice that EWG determines increases consensus.

8.2 Why not use a terse syntax with a symbol other than `!`, such as `pre#` or `pre@` ?

The `!` suffix follows successful existing practice in other languages, where it is widely recognized as indicating asserted or checked operations (see discussion in §5.2). Other symbols are less familiar and pose a higher risk of syntactic or lexical conflicts, and therefore are not proposed here.

However, as mentioned, this proposal is open to adopting any syntax choice that EWG recommends.

8.3 Why not do this only for `pre`, leaving `post` and `contract_assert` unchanged?

Consistency reasons.

Limiting always-enforced contract assertions to `pre` conditions would **leave gaps in reliability** because `post` conditions and `contract_assert` could still be ignored if different semantics are chosen separately. By applying the same approach across all contract assertion types, we ensure **consistent enforcement**, prevent silent failures, and simplify reasoning about contract assertions in large codebases.

However, we are not opposed to proceeding with a smaller subset if that increases consensus in EWG.

8.4 What about support for indirect calls (pointers to functions and virtual functions)?

This paper does not address those features because they are outside the scope identified by EWG for this proposal and no EWG-approved design exists for them at this time.

This proposal adds no constraints beyond those already present in C++26 that would limit the design of future features. For example, C++26 Contracts already permit `pre` contract assertions to be enforced; this proposal merely allows that choice to be made at the level of an individual contract assertion.

8.5 How will this change evaluation semantics?

This proposal has minimal impact on evaluation semantics, not changing them, but allowing them to be configurable at source code level.

When evaluating a contract assertion, the current C++26 Working Draft specifies that “it is implementation-defined which evaluation semantics is used for any given evaluation of a contract assertion” ([basic.contract.eval] p2 (<https://eel.is/c%2B%2Bdraft/basic.contract#eval-2>)). This proposal allows users to specify, in code, the evaluation semantic for targeted contract assertions. For example, users can write `pre!` to specify that a precondition is evaluated using terminating evaluation semantics.

Please also check section §8.12.

8.6 How does this proposal impact the Standard Library?

No impact.

Users are not expected to need to distinguish between "normal" contract assertions and "always-enforced" contract assertions. For instance, when `x == 0`, `pre(x > 0)` and `pre!(x > 0)` would be reported with identical metadata into the contract-violation handler.

8.7 Does this paper address all the questions raised in P3912R0 [5], P3919R0 [6], and P3946R0 [7]?

Yes, we think so.

Most of these questions are explicitly addressed in this paper, either in the proposal sections (see §3, §4, and §5) and/or in the FAQ section (see §8).

The remaining questions are implicitly addressed by the paper's deliberately narrow scope: it proposes adding only syntax to select the existing enforce or quick-enforce semantics for an individual contract assertion (semantics already permitted by the current C++ Working Draft for any contract assertion), and it proposes no other changes or extensions to the C++26 Contracts facility.

However, other design choices proposed by EWG remain open for consideration.

8.8 What about requiring header dependency?

No. The "always-enforced" contract assertions *do not* need any library construct. This proposal just adds syntax tokens to C++26 — *NO* header dependency required.

However, if future C++ Standards *may* lead the syntax for "always-enforced" contract assertions — e.g., `pre!` to be expressed in terms of labels — e.g., `pre<terminating>`, the header might be required.

Other design choices proposed by EWG remain open for consideration. Currently, this paper *does not* propose a header dependency.

8.9 Will "always-enforced" contract assertions behave differently from the "normal" ones?

No. All rules from the existing C++26 Contracts facility apply for "normal" and "always-enforced" contract assertions the same way, except for the evaluation semantics (see §8.5).

This proposal does not change how contract assertions are evaluated. In particular, it does not affect how the predicate of a contract assertion is evaluated or other properties (such as constification, how many times it is evaluated, whether it is evaluated on the caller or callee side, exception translation, etc.).

8.10 What does it mean that "execution will not continue past the contract assertion" when an "always-enforced" contract assertion is violated?

In this paper, "execution will not continue past the contract assertion" means that execution does not proceed past the violated contract assertion (e.g., not calling guarded function for `pre! / pre<terminating> precondition`).

Q: Does this proposal guarantee program termination in the event of "always-enforced" contract assertions violation?

A: No. This proposal *does not mandate program termination*; this proposal mandates using **only** terminating semantics for "always-enforced" contract assertions (which in most cases also means "contract-terminate"):

- For "always-enforced" contract assertions with `quick-enforce` semantics, a violation results in "program is contract-terminated" as per [basic.contract.eval#10] (<https://eel.is/c++draft/basic.contract.eval#10>).
- For "always-enforced" contract assertions with `enforce` semantics, a violation results similarly, by default, in "program is contract-terminated" as per [basic.contract.eval#13] (<https://eel.is/c++draft/basic.contract.eval#13>).
 - Do note that **this does not apply** when having a user-installed contract-violation handler that transfers control by throwing an exception.

In both cases, execution of the program after the violated contract assertion is prevented, and the asserted condition is guaranteed to hold for any code executed subsequently. See [basic.contract.eval#16] (<https://eel.is/c++draft/basic.contract.eval#16>) for details: "*The terminating semantics terminate the program if execution would otherwise continue normally past a contract violation*".

This distinction is intentional and aligns with the existing C++26 Contracts model, while still providing a reliable mechanism for specifying "always-enforced" contract assertions.

8.11 How does tooling benefit from this proposal?

For a static analyzer or compiler frontend, knowing that a contract assertion is always enforced provides the guarantee that the subsequent code (e.g., the function body for a pre contract assertion) can be analyzed with the certainty that the condition holds.

Also, compiler optimizers can benefit from the extra information implicitly attached to "always-enforced" contract assertions.

This empowers static analyzers and compilers to:

- Assume the condition is true for the remainder of the control flow.
- Warn on dead code for subsequent checks of the same (or subsets of the) condition, as the analyzer knows they must have already been true.

8.12 Does this proposal change the semantics of the C++26 Contracts facility?

No. Aside from introducing additional source syntax, this proposal does not introduce any semantics beyond those already permitted by the C++26 Contracts facility.

The current C++ Working Draft says (basic.contract.eval#3

(<https://eel.is/c++draft/basic.contract.eval#3>)):

Recommended practice: An implementation should provide [...] the option to translate a program such that **all evaluations** of contract assertions use the **enforce semantic**. **By default, evaluations of contract assertions should use the enforce semantic.**

An implementation that follows this recommended practice, when operating in its default configuration, already satisfies the intent of this proposal. This paper does not propose any new program behaviour beyond those behaviours already permitted by the existing C++26 Contracts model.

The *only effect of this proposal* is to restrict the implementation's ability to weaken or disable the "default" from recommended practice for contract assertions that are explicitly marked as always enforced in source code.

8.13 Is the predicate of an "always-enforced" contract assertion evaluated exactly once?

One major source of confusion for the ABI discussion is the number of times that the predicate of a contract assertion can be evaluated (see, for example, P3919R0 (<https://wg21.link/P3919R0>) [6]). By the current C++26 Contracts facility, the predicate can be evaluated zero times, once, or even twice. *This proposal does not aim to change that behaviour.*

We understand that it is desirable to ensure that the predicate is not evaluated more than once - e.g., for performance reasons (see, for example, P3919R0 (<https://wg21.link/P3919R0>) [6]), but this proposal does not take any steps in that direction.

The caller and the callee of a function with an always-enforced precondition may each independently generate code to check the predicate. This results in behaviour identical to that of a normal contract precondition when the implementation chooses enforce semantics (i.e., uses the defaults specified in the recommended practice).

Guaranteeing that the predicate is evaluated exactly once may have ABI implications. However, if this issue is addressed in a future standard (e.g., C++29), the same resolution would naturally apply to always-enforced contract assertions without requiring any additional changes.

8.14 Is there any previous implementation experience?

Yes.

Example of previous attempts to propose "always-enforced" contract assertions can be seen in P3640R0 (<https://wg21.link/P3640R0>) [8], P1429R3 (<https://wg21.link/P1429R3>) [9] and P1332R0 (<https://wg21.link/P1332R0>) [10].

Do note that examples in this section follows P1429 syntax:

Case	Syntax
"normal" syntax	<code>[[pre: x >= 0]]</code>
"always-enforced" syntax	<code>[[pre check_never_continue: x >= 0]]</code>

P1429 model was fully implemented in GCC (https://gcc.gnu.org/onlinedocs/gcc-15.1.0/gcc/C_002b_002b-Dialect-Options.html#index-fcontracts):

Enable experimental support for the C++ Contracts feature, as briefly added to and then removed from the C++20 working paper (N4820). The implementation also includes proposed enhancements from papers P1290, P1332, and P1429. This functionality is intended mostly for those interested in experimentation towards refining the feature to get it into shape for a future C++ Standard.

Examples:

- Compiler Explorer: `pre vs pre check_never_continue` in builds with `ignore semantics` (<https://godbolt.org/z/4o3sxEW44>)
- Compiler Explorer: `mixing pre vs pre check_never_continue` (<https://godbolt.org/z/ervMe7qfc>)

9. References

[1] P3911R0 — Darius Neațu, Andrei Alexandrescu, Lucian Radu Teodorescu, Radu Nichita, "RO 2-056 6.11.2 [basic.contract.eval] Make Contracts Reliably Non-Ignorable", WG21.

<https://wg21.link/P3911R0> (<https://wg21.link/P3911R0>) (published: 2025-11-04)

[2] P2900R14 — Joshua Berne, Timur Doumler, Andrzej Krzemieński, et al., "Contracts for C++", WG21. <https://wg21.link/P2900R14> (<https://wg21.link/P2900R14>) (published: 2025-02-13)

[3] P3400R2 — Joshua Berne, "Controlling Contract-Assertion Properties", WG21.

<https://wg21.link/P3400R2> (<https://wg21.link/P3400R2>) (published: 2025-12-14)

[4] P3835R0 — John Spicer, Ville Voutilainen, Jose Daniel Garcia Sanchez, "Contracts make C++ less safe – full stop!", WG21. <https://wg21.link/P3835R0> (<https://wg21.link/P3835R0>)

(published: 2025-09-03)

[5] P3912R0 — Timur Doumler, Joshua Berne, Gašper Ažman, Oliver Rosten, Lisa Lippincott, Peter Bindels, "Design considerations for always-enforced contract assertions", WG21.

<https://wg21.link/P3912R0> (<https://wg21.link/P3912R0>) (published: 2025-12-15)

[6] P3919R0 — Ville Voutilainen, "Guaranteed-(quick-)enforced contracts", WG21.

<https://wg21.link/P3919R0> (<https://wg21.link/P3919R0>) (published: 2025-11-05)

[7] P3946R0 — Andrzej Krzemieński, "Designing enforced assertions", WG21.
<https://wg21.link/P3946R0> (<https://wg21.link/P3946R0>) (published: 2025-12-14)

[8] P3640R0 — Jonas Persson, "Make contracts reliable by default", WG21.
<https://wg21.link/P3640R0> (<https://wg21.link/P3640R0>) (published: 2025-03-17)

[9] P1429R3 — Joshua Berne, John Lakos, "Contracts That Work", WG21.
<https://wg21.link/P1429R3> (<https://wg21.link/P1429R3>) (published: 2019-07-23)

[10] P1332R0 — Joshua Berne, Nathan Burgers, Hyman Rosen, John Lakos, "Contract Checking in C++: A (long-term) Road Map", WG21. <https://wg21.link/P1332R0>
(<https://wg21.link/P1332R0>) (published: 2018-11-26)